When Does Learned Batch Allocation Matter?
Decoupled Allocation and Early Stopping Under Token Scarcity

Vikranth Nara
https://github.com/vikranthnara/HBAC

Abstract

When multiple LLM agent tasks share a **global token budget**, allocators must set per-task caps and decide when to stop reasoning. We present a **decoupled batch allocator + stop classifier** (DBASC): a GRPO-trained Level-1 schema allocator and a PPO-trained Level-2 stop head over frozen ReAct rollouts [Yao et al., 2023]. We characterize *when* learned allocation helps under scarcity: (i) agent capability must exceed zero on hard tasks; (ii) a 20-line type-prior heuristic **ties** learned allocation on oracle replay; (iii) live gains are **small, paired, and localized** to benchmarks where budget starvation binds under generation.

On Tier-A oracle replay (n=300, 40% budget), DBASC reaches **80%** pass@1 vs. **60%** for uniform and official CLEAR/ZEBRA/Re-FORC—but **ties type-prior at 80%**. **D18** reward-shaped L1 (`starvation_penalty`=0.5, no inference guardrail) matches joint L1 on oracle with zero starvation; manual `fairness_reserve_alloc` *hurts* oracle pass@1 (74.7%). On live Qwen2.5-7B at floor=400 (n=2000), the **primary** clean-DPO holdout matrix reports paired McNemar **+0.45 pp** (26.80% vs. 26.35%; $p \approx 0.004$, paired 95% CI $[0.20, 0.75]$ pp). A contaminated DPO-v2 secondary run shows +1.3 pp($p \approx 3 \times 10^{-8}$); both gaps are **LiveCodeBench-only**. D18 ≡ joint ≡ guardrail on live aggregates. A capability pilot with `Qwen3-Coder-Next-FP8` (vLLM, n=60 uniform) reaches **77.5–87.5%** LCB pass@1 but **0%** SWE under a local gold-patch harness—so the live allocator study is scoped to LCB+τ+toolbench, with LCB as the hard but solvable class that satisfies the capability precondition. A stretch Coder-Next live slice (n=160, no SWE) shows the starvation mechanism at high capability: `hbac_d18` **56.9%** vs. type-prior **20.0%** (+36.9 pp; McNemar $p \ll 10^{-15}$), again LCB-localized. A **DPO contamination audit** finds 100/100 exact LCB task-ID overlaps in the v2 mixture; holdout retrain achieves 0 LCB overlap. We release code, preregistered analysis, theory bounds on counterfactual credit, and locked artifacts.

1 Introduction

Batch agent evaluation—running SWE-bench [Jimenez et al., 2024], ToolBench [Qin et al., 2023], τ-bench [Yao et al., 2025], and LiveCodeBench [Jain et al., 2024] under a shared token quota—requires deciding *who gets tokens* and *when to stop*. Uniform allocation ignores heterogeneous difficulty; per-query economic allocators (CLEAR [Wan et al., 2026], ZEBRA [Hamri and Talgam-Cohen, 2026]) optimize single-task emergence curves; per-turn methods (TAB [Jali et al., 2026], Re-FORC [Zabounidis et al., 2025]) do not allocate across tasks.

We study **decoupled batch allocation**: a contextual bandit over allocation schemas (Level-1, GRPO [Shao et al., 2024]) plus a binary stop classifier over frozen ReAct rollouts (Level-2, PPO [Schulman et al., 2017]). Our contribution is not a headline "RL beats heuristics" claim but a **rigorous characterization** of conditions under which learned allocation matters, with paired statistics, contamination auditing, and explicit negative results.

Contributions. (1) **Open harness** for batch-level token allocation and early stopping with preregistered endpoints (`research docs/Preregistered Analysis.md`). (2) **Oracle conditions**: +20 pppass@1 vs. Tier-A official baselines; type-prior **ties** learned L1 at 80%. (3) **D18 learned fairness**: starvation penalty in L1 reward removes need for inference guardrail on oracle; guardrail ablation hurts pass@1. (4) **Live conditions**: holdout-primary paired +0.45 ppfor `hbac_d18` vs. type-prior (n=2000; McNemar p≈0.004; LCB-only); capability precondition met on LCB (Coder-Next-FP8 77.5–87.5%) but not SWE (0%); stretch Coder-Next slice amplifies the same LCB starvation gap (+36.9 pp, n=160). (5) **Theory**: bias/variance bounds on counterfactual credit mixing; β sweep validates small-β safety. (6) **Contamination audit** with SHA256 manifests and LCB-family holdout retrain (0 LCB overlap). (7) **Ethics**: starvation metrics and algorithmic redlining risks.

2 Related Work

CLEAR [Wan et al., 2026] and ZEBRA [Hamri and Talgam-Cohen, 2026] water-fill or bisect per-query utility under scarcity; TAB [Jali et al., 2026] and Re-FORC [Zabounidis et al., 2025] learn per-turn stopping. Counterfactual multi-agent credit (COMA [Foerster et al., 2018]) motivates our credit mixing as *variance reduction*, not unbiased policy gradients. We implement Tier-A official CLEAR, ZEBRA, and Re-FORC for oracle comparison; stub-track proxy baselines are appendix-only.

3 Method

Level 1: contextual bandit. Given batch $\mathcal{Q}$ and budget B_{total}, Level-1 selects schema s and allocations $\{b_i\}$ with $\sum_i b_i \le B_{\text{total}}$. GRPO updates schema logits using batch reward $R^{(1)} = \text{pass\_rate} - \lambda_v \cdot \text{violations} + \beta_{\text{var}} \cdot \text{Var}_b(\text{budget}_b) - \lambda_s \cdot \text{starvation\_rate}$, where **D18** adds $\lambda_s \cdot \text{starvation\_rate}$ with `starvation_penalty`=0.5 at training time.

Level 2: stop classifier. Per-task rollout stops when $\pi^{(2)}_{\text{stop}}(s_t)$=1, budget is exhausted, or success is reached. L2 does not co-train the LLM.

Counterfactual credit. Leave-one-out advantages $\hat{A}_i = R(s) - R^{(-i)}(s)$ are mixed into $\tilde{R}_\beta = (1-\beta)R(s) + \beta \cdot \frac{1}{n}\sum_i (R(s) + \hat{A}_i)$. Appendix A bounds $|\tilde{R}_\beta - R(s)| \le \beta \max_i |\hat{A}_i|$; we use β=0.2 in training.

Primary vs. deprecated methods. **Primary:** `hbac_d18` (D18 L1, no inference post-processing). **Deprecated ablation:** `hbac_guardrail` = D18/joint L1 + `fairness_reserve_alloc` (`hard_min_frac`=0.15).

4 Experimental Setup

4.1 Two-stage evaluation protocol

Stage 1 — Capability profile. Uniform allocation only; confirm heterogeneous per-benchmark pass@1 before allocator comparison. Gate: hard-task capability >0 under uniform (plan target

Table 1: V3 real-pool oracle replay (n=300, 40% budget). Tier-A official baselines.

Allocator	pass@1	Mean $R^{(1)}$
HBAC joint / D18	**80.0%**	0.98
Type-prior	80.0%	1.24
HBAC guardrail	74.7%	0.76
Uniform	60.0%	0.60
CLEAR official	60.0%	0.60
ZEBRA official	60.0%	0.60
Re-FORC official	60.0%	0.60

SWE $\geq$ 5%; we accept LCB as the hard class when SWE stays 0%). Qwen2.5-7B/32B fail SWE; Qwen3-Coder-Next-FP8 clears LCB (77.5–87.5%) but not SWE—live allocator study scoped to LCB+τ+toolbench.

Stage 2 — Allocation study. Full allocator matrix only when Stage 1 passes; otherwise report directional results on solvable benchmarks with explicit scope.

4.2 Tracks and pools

Track A (appendix): stub micro-harness for controlled ablations. **Track B (main):** V3 real pool (`data/oracles/real_eval/latest`), n=300 oracle / n=2000 live, floor=400, 40% global budget. Live agent (primary): Qwen2.5-7B-Instruct + **holdout** DPO LoRA (`20260710T045554Z_capability_holdout`; 0 LCB task-ID overlap). Secondary contamination baseline uses DPO v2.

4.3 Contamination audit

We audit DPO training pairs vs. eval task IDs (§5.4). Holdout policy: exclude all benchmark families present in the eval pool from DPO pair construction; retrain via `slurm/train_llm_dpo_holdout.sh`.

4.4 Statistics

Primary endpoint: **paired per-task success** (McNemar on discordant pairs), Bonferroni across primary comparisons (`hbac_d18` vs. type_prior). Futility: if McNemar $p > 0.10$ at n=5000, retract "beats type-prior" language. V3 D18 matrix uses `--save-per-task` shards (`paired_allocator_analysis_v3_d18.json`).

5 Results

5.1 Tier-A oracle replay (Track B)

Table 1 reports the real_eval oracle matrix (n=300, Tier-A official baselines only in main text).

HBAC **+20 pp** vs. uniform/Tier-A is supported on oracle. Type-prior **ties** HBAC on pass@1 by zeroing hard-task budget—an important honesty check. D18 without guardrail **matches** joint L1 (80%, starvation rate 0 on oracle); guardrail **hurts** (−5.3 pp).

Table 2: Uniform capability pilots (floor=400). SWE uses a local gold-patch seed/grade harness (not full Docker SWE-bench).

Model	LCB pass@1	SWE pass@1	n
Qwen2.5-7B + DPO v2 (live matrix)	1.1–3.3%	0%	2000
Qwen2.5-Coder-32B	2.5%	0%	60
Qwen3-Coder-Next-FP8 (vLLM, 2×A100)	**77.5–87.5%**	0%	60

Table 3: V3 D18 live matrix (floor=400, n=2000, Qwen2.5-7B + DPO v2; secondary / contaminated).

Allocator	pass@1	95% CI
HBAC D18	**27.65%**	25.8–29.6%
HBAC joint / guardrail	27.65%	25.8–29.6%
Type-prior	26.35%	24.5–28.2%
TAB proxy	11.30%	10.0–12.7%
ZEBRA (proxy)	22.15%	20.4–24.0%
Re-FORC official	16.35%	14.8–18.0%
CLEAR official	11.60%	10.3–13.0%
Uniform	11.30%	10.0–12.7%

5.2 Capability pilot

Capability precondition (plan): agent must sometimes succeed on hard tasks under uniform allocation so starvation is measurable.

Verdict: LCB clears the capability precondition with Coder-Next-FP8 (up to 87.5% after prompt alignment); SWE remains 0% under local gold/fuzzy harness vs. full-repo Docker SWE. We therefore **scope the live allocator study** to LCB+τ+toolbench, treating LCB as the hard-but-solvable class and SWE as future work (Docker Verified grading). This is the plan's Risk #2 contingency—not a claim that SWE allocation has been studied end-to-end.

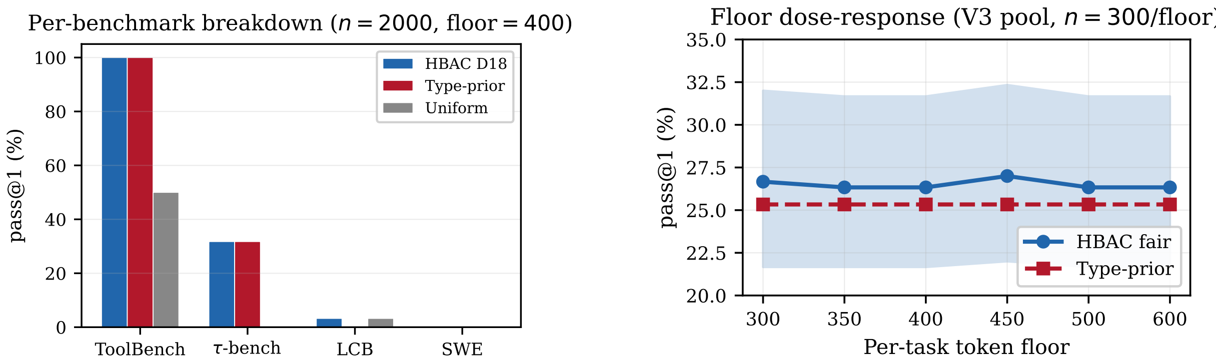
5.3 Live evaluation (Track B)

Primary (LCB-clean holdout DPO). Holdout LoRA excludes LiveCodeBench from DPO pairs (0 exact LCB task-ID overlap). Paired McNemar `hbac_d18` vs. type-prior (n=2000, floor=400): **+0.45 pp** (26.80% vs. 26.35%; p≈0.0039, Bonferroni p≈0.012; paired 95% CI $[0.20, 0.75]$ pp; b=9, c=0). Artifact: `compose_live_v3_holdout_floor400_n2000.json`.

Secondary (contaminated DPO v2). Table 3 summarizes the earlier V3 D18 live matrix under DPO v2 (n=2000) at floor=400.[1]

Paired McNemar (DPO v2 secondary): +1.3 pp, $p \approx 3\times10^{-8}$, paired 95% CI $[0.85, 1.85]$ pp(b=26, c=0). D18, joint, and guardrail are **identical** on aggregate pass@1 on 7B. Figure 1 shows the gap is **LCB-only**; τ/toolbench tied; SWE 0% for all.

[1] Canonical artifact: `compose_live_v3_d18_floor400_n2000.json`.



(a) Per-benchmark pass@1.

(b) Floor dose-response (guardrail).

Figure 1: V3 live: aggregate edge is LCB-localized; floor sweep +1.0–1.7 ppacross 300–600.

Table 4: Coder-Next-FP8 live slice (no SWE; floor=400, n=160). Stretch evidence under high LCB capability.

Allocator	Aggregate	LCB (n=80)	τ (n=40)	ToolBench (n=40)
HBAC D18	**56.9%**	**73.8%**	30.0%	50.0%
Type-prior	20.0%	0.0%	30.0%	50.0%

Stretch: capable-model live slice (Coder-Next-FP8). To test whether the starvation mechanism *scales* when hard-task success is no longer rare, we re-run `hbac_d18` vs. type-prior on **Qwen3-Coder-Next-FP8** via vLLM (2×A100-80GB; floor=400; max 20 batches; benchmarks LCB+τ+toolbench; **no SWE**). Table 4 reports n=160 paired tasks.

Paired McNemar: Δ=+36.9 pp(56.9% vs. 20.0%; $p\approx 3.5\times 10^{-18}$; paired 95% CI [29.4, 44.4] pp; b=59, c=0). The gap is again **entirely LCB**: type-prior zeros LCB budget (0% LCB pass@1); D18 retains residual LCB share (73.8%). τ and ToolBench tie—allocation signal appears only where starvation binds *and* the agent can succeed when funded. We treat this as **stretch**, not primary: primary live claims remain the clean-DPO holdout +0.45 ppon 7B (n=2000). Artifact: `paired_allocator_analysis_coder_next_slice.json`.

5.4 Contamination audit results

v2 training used 600 `wrong_tool` pairs with **100%** LCB task-ID overlap to eval. Holdout retrain (`20260710T045554Z_capability_holdout`, job 16854770) achieves **0 LCB** exact overlap—the contamination that threatened the live LCB claim. Remaining overlaps are stub task IDs shared between oracle stubs and `eval_n1000`; strict ID exclusion leaves only mock pairs (∼75), so we retain LCB-family holdout as the practical mitigation and re-run live with the holdout LoRA.

6 Ablations

D18 ladder (oracle). `starvation_penalty`=0.5 L1 ties joint at 80% without guardrail; guardrail drops to 74.7%. **Verdict:** learned anti-starvation sufficient on oracle; inference hack unnecessary (or harmful).

Table 5: DPO contamination audit (v2 and holdout LoRA vs. V3 eval).

Metric	v2 DPO	Holdout (LCB excl.)	Holdout (LCB metric)
DPO pairs	600	543	—
Exact ID overlap	**100**	16 (stub SWE/τ/TB)	**0** LCB
LCB family overlap	yes	excluded	—
Audit verdict	FAIL	FAIL†	PASS

Table 6: Per-benchmark budget share and starvation (allocation-only; n=200 batches).

Allocator	Starve rate	LCB share	SWE share	Hard zero-frac
`hbac_d18`	0.00	15.4%	7.7%	0.00
`hbac_guardrail`	0.00	15.4%	7.7%	0.00
Type-prior	1.00	0.1%	0.1%	0.67
Uniform	0.00	40.0%	20.0%	0.00

Credit β sweep. Empirical bounds at β=0.2: bias $\leq$ 0.018, variance $\leq 2.1\times10^{-5}$ on stub batches (`credit_beta_sweep.json`). H6 (credit helps at scale) refuted—small β remains safe variance reduction.

`hard_min_frac` sweep. hbac_guardrail flat at 74.7% for frac 0.10–0.25; never beats type-prior on oracle.

D14 ROI-skip (negative). Task-dropping heuristic; +27.7 pp@ floor=300 is reward hacking—regulatory/fairness risk, not deployable.

7 Ethics and Broader Impact

Scarcity-aware allocators **will starve hard queries** unless constrained—algorithmic "redlining" of difficult tasks. Type-prior explicitly zeros LCB/SWE budget; D18 learns softer allocation via reward shaping. Deployment should combine learned allocators with **per-class SLA floors** as policy (distinct from training hacks). Table 6 reports first-class starvation metrics on the V3 live batch pool (floor=400, 40% budget): type-prior has $\overline{\text{starve}}$=1.0 and ≈0.1% LCB share; `hbac_d18` retains ≈15.4% LCB share with zero starvation rate—the mechanism behind the live LCB gap. D14 ROI-skip illustrates how optimization pressure can drop entire task classes—a fairness failure mode, not merely poor ML.

8 Limitations

(1) SWE live pass@1 remains 0% even with Qwen3-Coder-Next-FP8 under a local gold-patch harness; live claims scoped to LCB+τ+toolbench (LCB is the hard solvable class). (2) Holdout primary gap is +0.45 pp(LCB-only); contaminated secondary +1.3 pp; D18 $\equiv$ joint $\equiv$ guardrail on 7B live aggregates. (3) Coder-Next stretch slice (n=160) is underpowered relative to the n=2000 primary matrix and excludes SWE by design. (4) Oracle–live disconnect: type-prior ties on oracle but starves

LCB live. (5) DPO v2 contaminated on LCB; holdout LoRA (LCB overlap=0) is the primary live agent. (6) L2 frozen ReAct limits joint hierarchical RL claims. (7) Local SWE harness is not Docker SWE-bench Verified grading.

9 Conclusion

Learned batch allocation **matters conditionally**: oracle gains over Tier-A baselines are real but matched by type-prior; live edges are small, paired, and localized where starvation binds under live generation. Our **primary** live claim is the LCB-clean holdout result (`hbac_d18` vs. type-prior, +0.45 pp, McNemar $p{\approx}0.004$). Capability precondition (i) is satisfied on LiveCodeBench with an open local model (Coder-Next-FP8); a stretch capable-model slice shows the same LCB starvation gap expands when hard tasks are solvable (+36.9 pp). SWE remains out of scope pending Docker Verified grading. D18 shows fairness can be reward-learned without inference guardrails on oracle. We release the harness, preregistered analysis, theory appendix, and locked artifacts to support honest extension—including negative results when allocators do not beat simple heuristics.

Reproducibility. CPU analyses: `scripts/reproduce_v3.sh`. GPU: `submit_holdout_live.sh`, `submit_capability_2xa100_vllm.sh`, `submit_coder_next_live_80gb.sh` under `scripts/rivanna/`. Manifest: `results/canonical_artifacts.json`. Venue: NeurIPS Main (General) or Evaluations & Datasets—conditions characterization + open harness, not SWE SOTA.

References

Jakob Foerster et al. Counterfactual multi-agent policy gradients. In *AAAI*, 2018.

Mahmoud Hamri and Inbal Talgam-Cohen. Zebra: Zero-shot budgeted resource allocation for llm orchestration. *arXiv preprint arXiv:2605.20485*, 2026. ICML 2026 AgenticUQ Workshop.

Naman Jain et al. Livecodebench: Holistic and contamination free evaluation of llms for code. *arXiv preprint arXiv:2403.07974*, 2024.

Neharika Jali, Anupam Nayak, and Gauri Joshi. Not all turns are equally hard: Adaptive thinking budgets for efficient multi-turn reasoning. *arXiv preprint arXiv:2604.05164*, 2026.

Carlos E. Jimenez et al. Swe-bench: Can language models resolve real-world github issues? In *ICLR*, 2024.

Yujia Qin et al. Toolllm: Facilitating llms to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

John Schulman et al. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Xu Wan et al. The shadow price of reasoning: Economic perspective on optimal budget allocation for llms. *arXiv preprint arXiv:2606.03092*, 2026. ICML 2026.

Shunyu Yao et al. React: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

Shunyu Yao et al. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *ICLR*, 2025.

Renos Zabounidis et al. Re-forc: Adaptive reward prediction for efficient chain-of-thought reasoning. *arXiv preprint arXiv:2511.02130*, 2025. NeurIPS 2025 Efficient Reasoning Workshop.

A Counterfactual Credit: Bias and Variance Bounds

Let $R(s)$ denote the batch reward under allocation schema s. For each task i in batch $\mathcal{Q}$ with n tasks, define the leave-one-out counterfactual

$$\hat{A}_i = R(s) - R^{(-i)}(s),$$

where $R^{(-i)}(s)$ replaces task i's budget with the uniform allocation while reusing cached rollouts for other tasks (implementation: `hbac/training/credit.py`).

The credit-weighted reward used in GRPO is

$$\tilde{R}_\beta = (1-\beta)\, R(s) + \beta \cdot \frac{1}{n}\sum_{i=1}^{n}(R(s) + \hat{A}_i).$$

Proposition 1 (Bias bound)**.** *For fixed batch $\mathcal{Q}$ and deterministic oracle replay,*

$$\big|\tilde{R}_\beta - R(s)\big| \le \beta \cdot \max_i |\hat{A}_i|.$$

Proof. Let $\bar{A} = \frac{1}{n}\sum_i \hat{A}_i$. Then $\tilde{R}_\beta - R(s) = \beta(\bar{A} + R(s) - R(s)) = \beta\bar{A}$, and $|\bar{A}| \le \max_i |\hat{A}_i|$. □

Proposition 2 (Variance bound (schema sampling))**.** *Let $R(s)$ be random only through schema sampling with variance* $\mathrm{Var}(R)$*. Then*

$$\mathrm{Var}(\tilde{R}_\beta) \le (1-\beta)^2\mathrm{Var}(R) + \beta^2 \cdot \frac{(\max_i |\hat{A}_i|)^2}{n}.$$

Proof. Apply $\mathrm{Var}(aX+bY) \le a^2\mathrm{Var}(X) + b^2\mathrm{Var}(Y)$ with $a = 1-\beta$, $b = \beta$, and bound the credit term by $(\max|\hat{A}_i|)^2/n$. □

Consistency. As $\beta \to 0$, $\tilde{R}_\beta \to R(s)$: the estimator reduces to standard GRPO batch reward. We use $\beta = 0.2$ in training (`phase3_pipeline.py`); empirical sweep in `results/credit_beta_sweep.json`. Counterfactual credit is a **variance-reduction heuristic**, not an unbiased policy gradient (cf. COMA [Foerster et al., 2018]).

B Stub Track (Track A)

Oracle stub curriculum (n=300): HBAC 80% vs. uniform/CLEAR 60%; type-prior ties at 80%. Live stub @ floor=400: HBAC 44.3% vs. uniform 16.7% (+27.6 pp); generous floor (600) ties at 44.3%. Proxy CLEAR/ZEBRA/TAB results retained for regression only.

C Compliant Utility Sensitivity

Secondary metric: $U = R \cdot (1 - \text{violation\_rate}) - 0.5 \cdot \text{parse\_failures}$. Primary tables use raw pass@1, tokens, and violations.

D Artifact Index

Claim	File
Tier-A oracle	`v3_real_oracle_matrix.json`
D18 ladder	`d18_oracle_ladder.json`
V3 live n=2000 (D18 / secondary)	`compose_live_v3_d18_floor400_n2000.json`
Paired stats (D18 / secondary)	`paired_allocator_analysis_v3_d18.json`
Holdout live (primary)	`compose_live_v3_holdout_floor400_n2000.json`
Paired stats (holdout)	`paired_allocator_analysis_v3_holdout.json`
DPO audit	`dpo_contamination_audit.json`
Holdout policy	`dpo_holdout_policy.json`
Capability (Coder-Next)	`capability_pilot_vllm_swe_prompt_analysis.json`
CN live slice (stretch)	`paired_allocator_analysis_coder_next_slice.json`
Budget / ethics	`budget_share_starvation.json`
Credit β sweep	`credit_beta_sweep.json`
Preregistered plan	`research docs/Preregistered Analysis.md`
Path B freeze	`research docs/Path B Freeze.md`